UNITÀ
00
LEZIONE
02

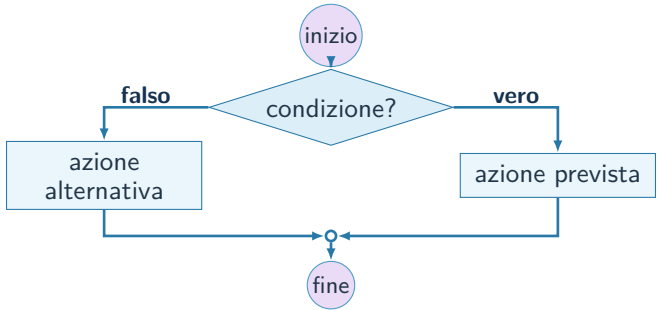
Condizioni e indentazione

Scegliere quali istruzioni eseguire

Prof. Francesco Gobbi
Anno scolastico 2026/2027
I.I.S.S. Galileo Galilei – Ostiglia (MN)

Il diagramma di flusso

Una **selezione** verifica una **condizione** e sceglie un percorso. Nei nostri diagrammi il **ramo vero** esce a destra, quello **falso** a sinistra. Il **raccordo** riunisce i percorsi prima di proseguire.



Condizioni e valori booleani

CODICE PYTHON

```
punteggio = 72
superata = punteggio >= 60

print(superata)
print(type(superata))
```

OUTPUT

```
True
<class 'bool'>
```

Il **confronto** `72 >= 60` produce `True`.

La **variabile** `superata` si riferisce a un oggetto di tipo `bool`.

Una **condizione** è un'espressione di cui Python verifica il valore di verità. Nei nostri esempi è un confronto oppure una combinazione di confronti.

I confronti

I confronti seguenti restituiscono `True` oppure `False` quando si applicano ai numeri dei nostri esempi.

Operatore	Significato	Esempio	Risultato
`==`	uguale a	`7 == 7`	`True`
`!=`	diverso da	`7 != 7`	`False`
`>`	maggiore di	`7 > 5`	`True`
`<`	minore di	`7 < 5`	`False`
`>=`	maggiore o uguale	`7 >= 7`	`True`
`<=`	minore o uguale	`7 <= 5`	`False`

Assegnazione e confronto

CODICE PYTHON

```
voto = 6
uguale = voto == 6

print(voto)
print(uguale)
```

OUTPUT

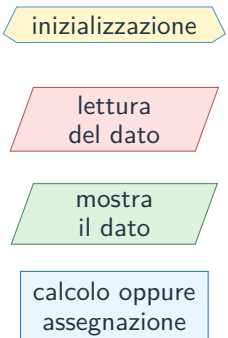
```
6
True
```

= associa un'**etichetta** a un oggetto.

== confronta due valori.

Nella seconda riga, Python valuta prima `voto == 6`, poi associa `uguale` al risultato booleano.

Notazione per dati e operazioni



Notazione adottata nel corso

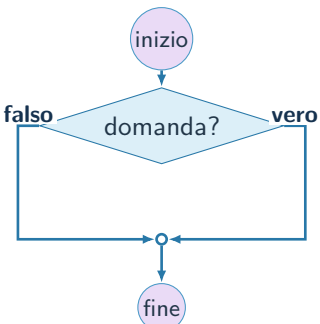
Esagono: **inizializzazione**, cioè assegnazione di un valore iniziale a una variabile. È la convenzione grafica del corso, non una dichiarazione preventiva.

Parallelogramma rosso: **input**, lettura di un dato.

Parallelogramma verde: **output**, visualizzazione di un risultato.

Rettangolo: operazione, calcolo o **assegnazione**.

Notazione per condizioni e raccordi



Un solo percorso per ogni verifica

I **cerchi** con “inizio” e “fine” delimitano il programma.

Il **rombo** contiene una **condizione**: si usa per le selezioni e, nelle prossime lezioni, per le condizioni delle iterazioni.

Ogni rombo ha il **ramo vero a destra** e il ramo **falso a sinistra**, entrambi etichettati.

Il piccolo **cerchio di raccordo** indica dove i percorsi alternativi si riuniscono. Non esegue operazioni e non attende altri rami: a ogni scelta viene seguito un solo percorso.

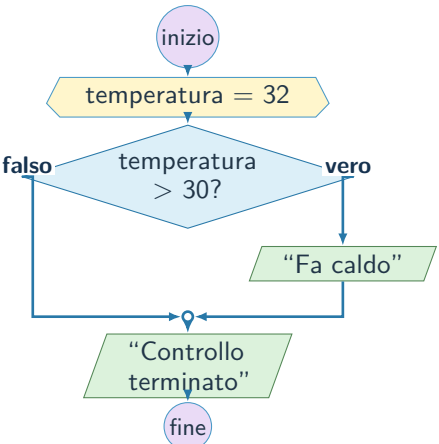
La selezione semplice: if

Un blocco facoltativo

La **condizione** controlla se la temperatura supera 30.

Il ramo **vero** stampa l'avviso; il ramo **falso** salta quella stampa.

Dopo il **raccordo**, il messaggio finale compare in entrambi i casi.



L'indentazione in Python

L'**indentazione** è lo spazio all'inizio di una riga. In Python raggruppa le istruzioni nei **blocchi** e rappresenta la loro struttura. Dopo una riga come `if condizione:`, le istruzioni del blocco devono avere un livello di **indentazione** maggiore. Il ritorno al livello esterno segna la fine del blocco.

In questa lezione usiamo **quattro spazi per ogni livello**, come raccomanda lo stile Python. La regola sintattica è la coerenza dei livelli: **quattro spazi** sono una convenzione, non l'unica quantità accettata dal linguaggio.

Indentazione e appartenenza a un blocco

```
temperatura = 32
if temperatura > 30:
    print("Fa caldo")
    print("Porta dell'acqua")
print("Controllo terminato")
```

Stessa indentazione, stesso blocco

Le due righe rientrate di **quattro spazi** dipendono dalla condizione.

L'ultima riga torna al **livello esterno**.

Il blocco indentato viene eseguito quando la condizione è **vera**. La stampa finale viene eseguita anche quando è **falsa**.

Esecuzione di if con condizione vera

CODICE PYTHON

```
temperatura = 32
if temperatura > 30:
    print("Fa caldo")
print("Controllo terminato")
```

OUTPUT

```
Fa caldo
Controllo terminato
```

`if` introduce la **condizione**; i due punti `:` terminano l'intestazione. Le righe indentate formano il **blocco**.

La prima chiamata a `print()` è indentata e dipende dal **confronto**. La seconda appartiene al livello di **indentazione** esterno ed è successiva alla selezione.

Esecuzione di if con condizione falsa

CODICE PYTHON

```
temperatura = 24
if temperatura > 30:
    print("Fa caldo")
print("Controllo terminato")
```

OUTPUT

```
Controllo terminato
```

24 > 30 è **falso**: Python non esegue la prima chiamata a `print()`.

Il programma continua dalla prima istruzione esterna al **blocco**. La **condizione** falsa non termina il programma.

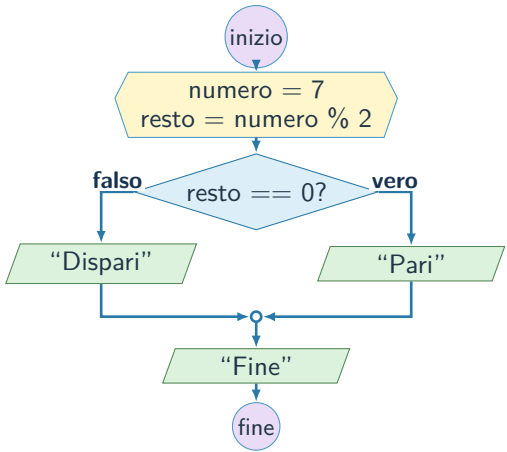
La selezione alternativa: if-else

Due alternative

Il **resto** della divisione per 2 determina la parità.

Il ramo **vero** stampa “Pari”; il ramo **falso** stampa “Dispari”.

`else` raccoglie il caso falso. Dopo il **raccordo**, la stampa “Fine” è comune.



Verifica della parità di un numero

CODICE PYTHON

```
numero = 7
resto = numero % 2

if resto == 0:
    print("Pari")
else:
    print("Dispari")
print("Fine")
```

OUTPUT

```
Dispari
Fine
```

% calcola il resto della divisione intera.

Per 7 e 2 il resto è 1.

`resto == 0` è **falso**: si esegue il blocco associato a `else`. Con `numero = 8`, l'**output** sarebbe "Pari", poi "Fine".

Allineamento dei blocchi if ed else

CODICE PYTHON

```
voto = 7
if voto >= 6:
    esito = "Sufficiente"
else:
    esito = "Insufficiente"
print(esito)
```

L'allineamento dei rami

`if` ed `else` iniziano nella stessa colonna.

Ogni **ramo** contiene un **blocco** indentato di **quattro spazi**.

`print(esito)` torna al livello della scelta e si esegue dopo il ramo selezionato.

Assegnazione nei rami condizionali

CODICE PYTHON

```
voto = 4
if voto >= 6:
    esito = "Sufficiente"
print(esito)
```

ERRORE

```
NameError
```

In una sessione nuova, il **ramo** non viene eseguito e alla variabile `esito` non è ancora associato alcun oggetto.

La soluzione della slide precedente assegna `esito` in entrambi i rami. Nei notebook, un vecchio valore può nascondere il problema: verifica anche dopo un riavvio della sessione.

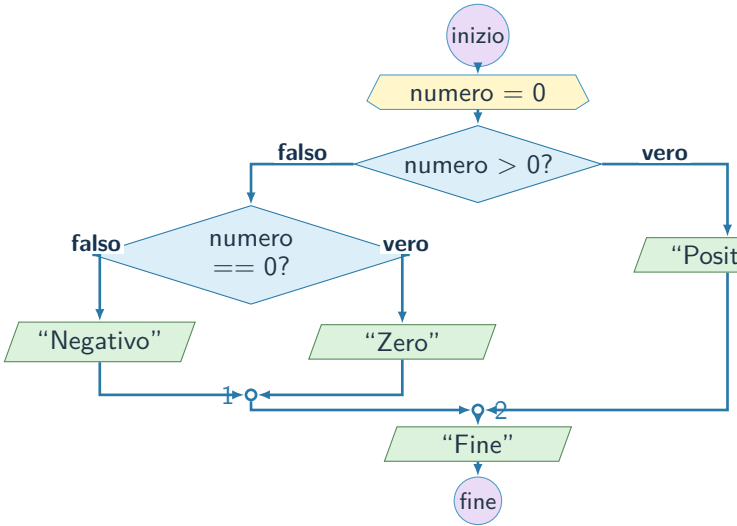
La selezione multipla: if-elif-else

Una catena `if-elif-else` sceglie tra più casi alternativi. Python valuta le condizioni nell'ordine in cui sono scritte.

- `if` introduce la prima **condizione**.
- `elif` significa "altrimenti, se": la condizione viene valutata solo se tutte quelle precedenti sono false. Possono esserci più `elif`.
- Alla prima condizione **vera**, Python esegue il **blocco** corrispondente e salta le restanti condizioni della catena.
- `else`, se presente, raccoglie il caso in cui tutte le condizioni sono false. Senza `else`, può non essere eseguito alcun **ramo**.

`if`, `elif` ed `else` sono allineati. Ciascuno termina con due punti e ha un blocco indentato. Il programma prosegue dopo la catena.

Diagramma di flusso di if-elif-else



1: riunione dei casi Zero e Negativo.

2: riunione di tutti i casi prima della stampa comune.

Classificazione del segno di un numero

CODICE PYTHON

```
numero = 0
if numero > 0:
    print("Positivo")
elif numero == 0:
    print("Zero")
else:
    print("Negativo")
print("Fine")
```

OUTPUT

```
Zero
Fine
```

Il primo **confronto** è **falso**, quindi Python verifica l'`elif`.

Il secondo è **vero**: stampa "Zero" e non esegue il **blocco** `else`. I due punti e l'**indentazione** sono necessari per ogni **ramo**.

Percorsi di esecuzione di if-elif-else

Per il programma precedente, ogni valore porta a un percorso diverso.

numero	**numero > 0**	**numero $== 0$**	**Output**
5	**vero**	non valutato	Positivo, poi Fine
0	**falso**	vero	Zero, poi Fine
−3	falso	falso	Negativo, poi Fine

"Non valutato" è diverso da "falso": una volta scelto un **ramo**, Python non controlla le condizioni successive della stessa catena.

Ordine di valutazione delle condizioni

CODICE PYTHON

```
voto = 9
if voto >= 6:
    print("Sufficiente")
elif voto >= 8:
    print("Buono")
else:
    print("Insufficiente")
```

OUTPUT

```
Sufficiente
```

Anche se 9 è almeno 8, il primo **confronto** è già **vero**.

Il **ramo** "Buono" non può essere raggiunto: ogni voto almeno 8 soddisfa anche la prima **condizione**. Occorre ordinare diversamente le **soglie**.

Classificazione mediante soglie

CODICE PYTHON

```
voto = 9
if voto >= 8:
    esito = "Buono"
elif voto >= 6:
    esito = "Sufficiente"
else:
    esito = "Insufficiente"
print(esito)
```

OUTPUT

```
Buono
```

Per raggiungere l'`elif`, deve essere **falso** `voto >= 8`. Il suo **ramo** copre quindi i valori da 6 incluso a 8 escluso.

Qui supponiamo che il voto sia già valido. Più avanti controlleremo anche il suo intervallo.

Selezioni indipendenti e selezione multipla

CODICE PYTHON

```
voto = 9
if voto >= 6:
    print("Sufficiente")
if voto >= 8:
    print("Buono")
```

```
Sufficiente
Buono
```

CODICE PYTHON

```
voto = 9
if voto >= 6:
    print("Sufficiente")
elif voto >= 8:
    print("Buono")
```

```
Sufficiente
```

Due `if` sono due scelte indipendenti. Una catena `if-elif` seleziona al massimo un **ramo**. Usiamo più `if` quando più azioni possono essere necessarie insieme.

Gli operatori logici

Gli operatori logici permettono di combinare più verifiche.

Operatore	Significato con condizioni booleane
`and`	Il risultato è **vero** solo se entrambe le condizioni sono vere.
`or`	Il risultato è vero se almeno una **condizione** è vera, anche se lo sono entrambe.
`not`	Inverte il valore logico: vero diventa **falso** e viceversa.

Esempio: per accedere a un laboratorio servono sia l'autorizzazione sia il badge. La condizione è `autorizzato == True and badge == True`.

Le tabelle di verità

A	B	A and B	A or B
`False`	`False`	`False`	`False`
`False`	`True`	`False`	`True`
`True`	`False`	`False`	`True`
`True`	`True`	`True`	`True`

`not True` restituisce `False`; `not False` restituisce `True`. Le tabelle qui mostrano operandi booleani; in generale `and` e `or` restituiscono uno dei loro operandi.

L'operatore and

CODICE PYTHON

```
autorizzato = True
badge = False

if autorizzato == True and badge == True:
    print("Accesso consentito")
else:
    print("Accesso negato")
```

OUTPUT

```
Accesso negato
```

L'autorizzazione è presente, ma manca il badge. Una sola **condizione** vera non basta per `and`.

Impostando anche `badge = True`, il programma stampa "Accesso consentito".

Gli operatori or e not

CODICE PYTHON

```
abbonato = False
ospite = True

if abbonato == True or ospite == True:
    print("Ingresso consentito")
if not (abbonato == True):
    print("Nessun abbonamento")
```

OUTPUT

```
Ingresso consentito
Nessun abbonamento
```

Il primo `if` accetta almeno uno dei due requisiti.

Il secondo è una verifica indipendente: `not (False == True)` vale `True`. Perciò vengono eseguiti entrambi i blocchi.

Verifica di appartenenza a un intervallo

CODICE PYTHON

```
voto = 7
valido = 0 <= voto <= 10

if valido == True:
    print("Voto valido")
else:
    print("Fuori intervallo")
```

OUTPUT

```
Voto valido
```

`0 <= voto <= 10` verifica entrambi i limiti, inclusi gli estremi.

Con questi numeri equivale a `voto >= 0 and voto <= 10`. Per escludere gli estremi useremmo `0 < voto < 10`.

Differenza tra congiunzione e disgiunzione

CODICE PYTHON

```
colore = "rosso"
if colore == "rosso" and colore == "blu":
    print("Colore ammesso")
else:
    print("Colore escluso")
```

OUTPUT

```
Colore escluso
```

Un colore non può essere contemporaneamente “rosso” e “blu”.

Per ammettere uno dei due colori serve `or` tra i due confronti. Con `or`, l’**output** diventa “Colore ammesso”.

Confronti espliciti

Negli esempi rendiamo espliciti i **confronti** per seguire il ragionamento. Se una variabile fa riferimento a un oggetto `bool`, il confronto con `True` non è obbligatorio.

CODICE PYTHON

```
autorizzato = True
if autorizzato == True:
    print("Permesso presente")
```

OUTPUT

```
Permesso presente
```

Il confronto `True == True` è **vero**:

il **blocco** viene eseguito.

Con `autorizzato` booleano, `if autorizzato:` esprime la stessa scelta. Per negarla si può usare `if not autorizzato:`. Nei confronti composti, ogni alternativa va scritta per intero: `colore == "rosso" or colore == "blu"`.

Precedenza degli operatori logici

La **precedenza** stabilisce il raggruppamento: confronti, poi `not`, poi `and`, infine `or`. Il corto circuito può evitare la valutazione di una parte dell'espressione. Le parentesi rendono esplicito il raggruppamento desiderato.

Espressione	Risultato
`True or False and False`	`True`
`(True or False) and False`	`False`

Nella prima riga, `and` lega i due `False`. Nella seconda, le parentesi impongono di risolvere prima `True or False`. Per formule lunghe, usa etichette significative per i singoli requisiti.

Valutazione a "corto circuito"

CODICE PYTHON

```
divisore = 0
if divisore != 0 and 10/divisore > 2:
    print("Quoziente maggiore di 2")
else:
    print("Condizione non soddisfatta")
```

OUTPUT

```
Condizione non soddisfatta
```

Con and, se la prima condizione è falsa,

la seconda non viene valutata: qui la divisione per zero non avviene.

Con or, basta una prima condizione vera per non valutare la seconda. L'ordine può quindi essere importante.

Le condizioni annidate

CODICE PYTHON

```
autorizzato = True
badge = False
if autorizzato == True:
    if badge == True:
        print("Entra")
    else:
        print("Manca il badge")
else:
    print("Manca il permesso")
```

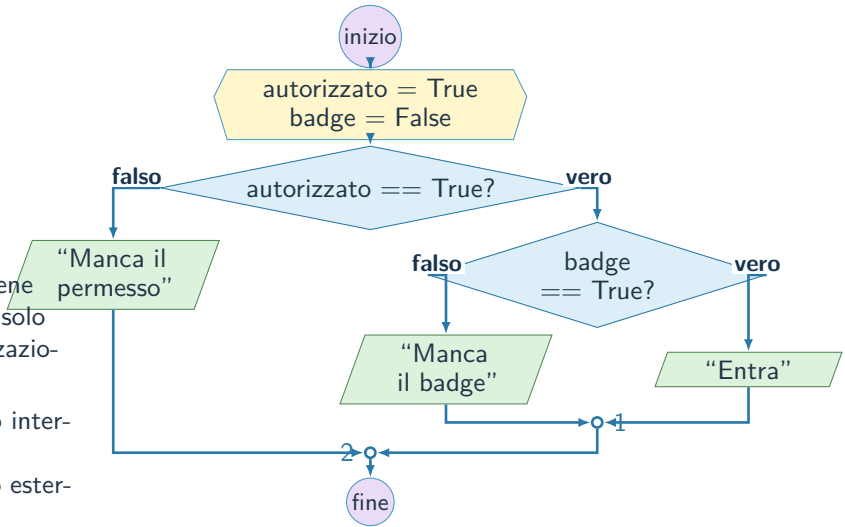
OUTPUT

```
Manca il badge
```

La verifica del badge avviene soltanto dentro il **ramo** dell'autorizzazione.

Il primo `else`, indentato di **quattro spazi**, appartiene all'`if badge == True`. Il secondo, al margine iniziale, appartiene all'`if autorizzato == True`.

Diagramma di flusso delle selezioni annidate



Il **badge** viene controllato solo se l'autorizzazione è vera.

1: raccordo interno.

2: raccordo esterno.

Errore: indentazione mancante

CODICE PYTHON

```
eta = 16
if eta >= 14:
print("Accesso consentito")
```

ERRORE

```
IndentationError
```

Dopo i due punti, Python si aspetta un **blocco** indentato. La chiamata a `print()` deve essere indentata rispetto a `if`; nel corso usiamo **quattro spazi**.

Il programma contiene un **errore di sintassi** e non viene eseguito. Il testo completo del messaggio può variare tra versioni.

Errore: indentazione non prevista

CODICE PYTHON

```
numero = 5
    print(numero)
```

ERRORE

```
IndentationError
```

La prima riga non apre alcun **blocco**.

L'**indentazione** della seconda istruzione non è quindi prevista.

Correzione: allinea `print(numero)` all'assegnazione. L'indentazione esprime la struttura dei blocchi del programma.

Errore: livelli di indentazione incoerenti

CODICE PYTHON

```
eta = 16
if eta >= 14:
    print("Accesso consentito")
  print("Fine")
```

ERRORE

```
IndentationError
```

L'ultima riga ha due spazi: non coincide né con il margine esterno né con il livello di **quattro spazi** del **blocco**.

Usa zero spazi se “Fine” deve comparire sempre; quattro se deve dipendere dalla **condizione**.

Tabulazioni e spazi nell'indentazione

Il carattere di tabulazione e lo spazio sono caratteri diversi. Due righe possono sembrare allineate nell'editor e presentare livelli di **indentazione** interpretati diversamente. Una combinazione incoerente di tab e spazi può causare `TabError`, un caso particolare di `IndentationError`.

Nel notebook usa **quattro spazi** per livello. Se copi codice da altre fonti, verifica l'indentazione e, quando disponibile, usa il comando dell'editor per convertire le tabulazioni in spazi.

Indentazione e significato del programma

CODICE PYTHON

```
eta = 12
if eta >= 14:
    print("Accesso consentito")
print("Fine")
```

```
Fine
```

CODICE PYTHON

```
eta = 12
if eta >= 14:
    print("Accesso consentito")
    print("Fine")
```

```
(nessun output)
```

Entrambi i programmi sono sintatticamente validi. A destra anche “Fine” dipende dalla **condizione** falsa. È un **errore logico** se l’intenzione era stampare quel messaggio in ogni caso.

Sintassi delle intestazioni e dei blocchi

- Ogni intestazione `if`, `elif` o `else` termina con `:`. Senza i due punti si ottiene un `SyntaxError`.
- `elif` richiede una **condizione**; `else` non la ammette. Per esempio, `else voto < 6:` non è valido.
- Un **blocco** deve contenere almeno un'istruzione. Una riga vuota o un commento da soli non bastano.

Durante la costruzione di un programma si può usare `pass` come istruzione provvisoria: non svolge alcuna azione, ma rende il blocco sintatticamente completo.

Acquisizione e confronto di dati numerici

CODICE PYTHON

```
eta = int(input("Eta: "))
if eta >= 14:
    print("Accesso consentito")
else:
    print("Accesso negato")
```

OUTPUT

```
Accesso consentito
```

Esempio di **input**: 16. `int()` converte il testo nel numero intero usato dal **confronto**.

Se l'utente inserisce un testo non convertibile, come "sedici", la conversione produce `ValueError` prima della scelta. Qui assumiamo un intero valido.

Diagramma di flusso con input numerico

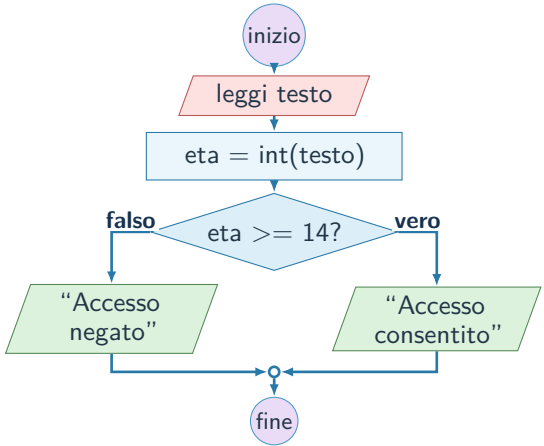
Lettura, conversione e scelta

`input()` legge una **stringa**; `int()` la converte in un intero.

La variabile `eta` fa riferimento al risultato. Il valore di verità della **condizione** determina il percorso da seguire.

Lo schema separa la **lettura** dalla **conversione**. Nel codice, entrambe avvengono nell'istruzione `eta = int(input("Eta: "))`.

Si assume un testo convertibile in intero.



Validazione e classificazione di un voto

CODICE PYTHON

```
voto = 8
if voto < 0 or voto > 10:
    esito = "Voto non valido"
elif voto >= 8:
    esito = "Buono"
elif voto >= 6:
    esito = "Sufficiente"
else:
    esito = "Insufficiente"
print(esito)
```

OUTPUT

```
Buono
```

Prima escludiamo i valori fuori intervallo.

Poi classifichiamo quelli validi, partendo dalla soglia più alta.

Ogni **ramo** assegna `esito`: la stampa finale può usare la **variabile** indipendentemente dal percorso scelto.

Verifica dei casi limite

Un solo esempio corretto non basta. Per il classificatore dei voti, proviamo ogni **ramo** e i valori vicini alle **soglie**.

Valore di voto	Risultato atteso
`-1, 11`	Voto non valido
`0, 5.9`	Insufficiente
`6, 7.9`	Sufficiente
`8, 10`	Buono

Nei notebook cambia il dato e riesegui l'intera cella. Se il codice è diviso in più celle, riesegui quelle dipendenti; un vecchio **output** non si aggiorna da solo.

Esercizio 1 — prevedere gli output

CODICE PYTHON

```
x = 5
if x > 0:
    print("A")
if x > 3:
    print("B")
else:
    print("C")
print("D")
```

Senza eseguire il codice:

1. Scrivi l'**output** per `x = 5`.
2. Ripeti con `x = 2` e `x = -1`.
3. Indica a quale `if` appartiene `else`.
4. Spiega perché "D" compare in tutti i casi.

Poi verifica le previsioni nel notebook.

Esercizio 2 — correggere l'indentazione

CODICE PYTHON

```
voto = 7
if voto >= 6:
print("Promosso")
    else:
        print("Da recuperare")
    print("Fine verifica")
```

Riscrivi il programma in modo che:

- mostri un solo esito in base al voto;
- stampi sempre "Fine verifica";
- usi **un'indentazione** per ogni livello.

Provalo con i valori 5, 6 e 9. Disegna i due percorsi del diagramma.

Esercizio 3 — tre fasce di temperatura

Acquisisci una temperatura con `float(input(...))`. Assumi che l'utente inserisca un numero valido.

- Sotto 10 gradi: mostra "Freddo".
- Da 10 incluso a 25 escluso: mostra "Mite".
- Da 25 in poi: mostra "Caldo".

Prima disegna il diagramma, poi scrivi una catena `if-elif-else`. Spiega perché la seconda **condizione** può essere soltanto `temperatura < 25`.

Verifica almeno questi dati: `-2`, `9.9`, `10`, `24.9`, `25`, `32`. Dopo la scelta stampa sempre "Analisi completata".

Esercizio 4 — autorizzazione e badge

Nel notebook Python assegna due valori booleani alle variabili `autorizzato` e `badge`.

1. Scrivi una versione con `and`: accesso consentito soltanto quando entrambi sono veri; negli altri casi, accesso negato.
2. Scrivi una versione annidata che distingua “Manca il permesso” da “Manca il badge”.
3. Prova tutte le quattro combinazioni della tabella di verità.

Confronta i due programmi: la decisione finale coincide, ma la versione annidata fornisce informazioni diverse sul motivo del rifiuto.

Riepilogo

Struttura	Quando usarla
`if`	Un'azione facoltativa, eseguita solo se la **condizione** è vera.
`if-else`	Due alternative che coprono **vero** e **falso**.
`if-elif-else`	Più casi alternativi, verificati in ordine.
Più `if`	Verifiche indipendenti; più blocchi possono essere eseguiti.
`if` annidati	Una verifica dipende dal risultato di una scelta esterna.

L'**indentazione** delimita i blocchi di istruzioni. Prima di eseguire il codice, individua le condizioni, i percorsi possibili e le istruzioni comuni dopo la scelta.